AN OPEN, PRACTICAL SYLLABUS

LLM INFERENCE ENGINEERING LEARNING PATH

The skills behind efficient LLM serving - from transformer fundamentals to GPU kernels, runtime internals, and distributed systems.

10 phases | 180 ordered topics | Practical milestones

Learn in sequence. Build working systems. Measure before optimizing.

Learning path at a glance

Follow the phases in order. Each phase builds on the concepts and practical skills developed in the previous one. The final two phases are advanced extensions; earlier phases establish the core inference-engineering foundation.

Phase	Focus	Topics
01	Deep Learning & LLM Foundations	001 - 025
02	LLM Inference Fundamentals	026 - 048
03	GPU Architecture & Performance	049 - 065
04	Inference Serving & Runtime Engineering	066 - 092
05	Production Inference Infrastructure	093 - 104
06	CUDA & Triton Kernel Engineering	105 - 120
07	Advanced Inference Optimization	121 - 134
08	Multi-GPU & Distributed Inference	135 - 148
09	Compilers & Accelerator Programming	149 - 162
10	Frontier AI Infrastructure	163 - 180

Suggested learning method

For each topic: understand the concept, implement a small example, test its correctness, and measure performance where relevant. The practice milestone at the end of each phase helps connect the topics into a usable skill.

Some advanced topics require hardware beyond a single GPU. Those can be explored using a cloud environment or by studying implementations and published measurements before running larger experiments.

Prerequisite guide

Core: Phases 1-5 cover model fundamentals, inference execution, GPU performance, serving runtimes, and service reliability. Deeper performance specialization: Phases 6-8 cover GPU kernels, advanced optimizations and distributed execution. Advanced extensions: Phases 9-10 cover compilers, alternative accelerators and frontier-scale infrastructure.

PHASE 01 / 10

Deep Learning & LLM Foundations

Build the model before optimizing its runtime.

PyTorch fundamentals

001	Tensors, shapes, strides, broadcasting
002	Matrix multiplication and tensor operations
003	Computational graphs and autograd
004	Forward and backward propagation
005	Gradient descent, SGD and AdamW
006	nn.Module, parameters and buffers
007	DataLoader, batching and training loops
008	CPU/GPU tensor transfers and device management
009	PyTorch memory management and mixed precision

Build a Transformer from scratch

010	Text tokenization, BPE and vocabulary construction
011	Token embeddings and positional embeddings
012	Query, Key and Value projections
013	Scaled dot-product self-attention
014	Causal attention masking
015	Multi-head attention
016	Multi-query and grouped-query attention (MQA/GQA)
017	LayerNorm and RMSNorm
018	Feed-forward networks and activation functions
019	Residual connections and transformer blocks
020	Rotary positional embeddings (RoPE)
021	GPT-style decoder-only model architecture
022	Cross-entropy loss and next-token prediction
023	Training a small GPT model
024	Autoregressive text generation and sampling
025	Model weights, checkpoints and serialization

PRACTICE MILESTONE Build and train TinyGPT entirely in PyTorch. Explain each module and trace the tensor shapes.

PHASE 02 / 10

LLM Inference Fundamentals

Understand decoding, memory and the cost of generating a token.

Understand LLM execution

026	Training versus inference
027	Transformer forward-pass execution
028	Prefill versus decode
029	Autoregressive generation loop
030	KV-cache fundamentals
031	Implement KV caching from scratch
032	KV-cache memory calculation
033	Attention computation complexity
034	Memory bandwidth versus compute bottlenecks
035	Model weights, activations and memory footprint
036	FP32, FP16, BF16, FP8 and INT8 representations
037	Sampling: temperature, top-k, top-p and greedy decoding

Inference performance fundamentals

038	Latency versus throughput
039	Time to first token (TTFT)
040	Time per output token (TPOT)
041	Inter-token latency
042	Tokens per second and requests per second
043	P50, P95 and P99 latency
044	Prefill/decode performance modeling
045	GPU utilization and memory utilization
046	Compute-bound versus memory-bound workloads
047	Arithmetic intensity and the roofline model
048	Benchmark warmups, synchronization and statistical validity

PRACTICE MILESTONE Implement an inference benchmark; compare token generation with and without KV caching.

PHASE 03 / 10

GPU Architecture & Performance

Develop hardware intuition and trustworthy profiling habits.

GPU hardware fundamentals

049	CPU versus GPU architecture
050	Streaming multiprocessors (SMs)
051	Threads, warps, blocks and grids
052	GPU execution and scheduling
053	Global, shared, local and register memory
054	L1/L2 caches and memory hierarchy
055	Memory coalescing and access patterns
056	Warp divergence and synchronization
057	Occupancy, register pressure and latency hiding
058	Tensor Cores and matrix operations

GPU profiling

059	CUDA execution and asynchronous operations
060	CUDA events and accurate timing
061	PyTorch Profiler
062	NVIDIA Nsight Systems
063	NVIDIA Nsight Compute
064	Kernel launch overhead and CPU-GPU synchronization
065	Bandwidth, occupancy and roofline analysis

PRACTICE MILESTONE Profile a transformer forward pass and identify its most expensive operations.

PHASE 04 / 10

Inference Serving & Runtime Engineering

Study the source and request lifecycle of real serving engines.

Introduction to inference servers

066	Hugging Face Transformers inference
067	Model loading and Safetensors
068	GPU memory allocation and out-of-memory handling
069	vLLM installation, serving and APIs
070	SGLang installation, serving and APIs
071	OpenAI-compatible inference APIs
072	Request concurrency and streaming responses
073	Workload generation and serving benchmarks

vLLM internals

074	vLLM V1 architecture
075	Engine and model runner
076	Scheduler architecture
077	Continuous batching
078	Token budgets and scheduling policies
079	PagedAttention
080	KV-cache blocks and memory allocation
081	Prefix caching and block reuse
082	Chunked prefill
083	Request preemption and cache eviction
084	CUDA graphs and execution overhead
085	Trace a request through vLLM source code

SGLang internals

086	SGLang SRT architecture
087	RadixAttention
088	Radix trees and prefix matching
089	HiCache and hierarchical KV caching
090	SGLang scheduling and continuous batching
091	Request routing and cache-aware scheduling
092	vLLM versus SGLang architecture and performance

PRACTICE MILESTONE Build a reproducible vLLM/SGLang benchmark suite and explain performance differences.

PHASE 05 / 10

Production Inference Infrastructure

Build robust serving systems, not just working demos.

Distributed service foundations

093	Python asyncio and concurrency
094	C++ systems programming fundamentals
095	Processes, threads, locks and synchronization
096	Linux memory, scheduling and I/O
097	HTTP, HTTP/2, gRPC and streaming
098	Queueing theory and Little's Law
099	Load balancing and backpressure
100	Request admission control and rate limiting
101	Timeouts, retries and cancellation
102	Multi-tenant inference serving
103	Metrics, tracing, logging and observability
104	SLOs, failure handling and performance regression testing

PRACTICE MILESTONE Build an inference gateway with concurrency control, streaming, load testing, metrics and reliability tests.

PHASE 06 / 10

CUDA & Triton Kernel Engineering

Write and optimize correct GPU programs using measurements.

CUDA programming

105	CUDA C++ programming model
106	Kernel launches and grid configuration
107	Global/shared memory access optimization
108	Thread synchronization and warp-level primitives
109	Parallel reductions and scans
110	Matrix multiplication tiling
111	Tensor Core programming fundamentals
112	Kernel fusion and numerical correctness

Triton programming

113	Triton language and JIT compilation
114	Block pointers, layouts and tensor operations
115	Triton vector addition and reduction
116	Triton softmax and layer normalization
117	Triton matrix multiplication
118	Autotuning and kernel performance measurement
119	FlashAttention algorithm and implementation principles
120	Fused attention kernels and optimization

PRACTICE MILESTONE Implement and benchmark CUDA/Triton kernels against library baselines and validate correctness across shapes.

PHASE 07 / 10

Advanced Inference Optimization

Explore the trade-offs that determine cost, throughput and response quality.

Model and runtime optimizations

121	Weight quantization: INT8, INT4, FP8
122	GPTQ, AWQ and quantization calibration
123	KV-cache quantization and compression
124	Quantization accuracy and performance trade-offs
125	Speculative decoding fundamentals
126	Draft-model and verifier execution
127	Speculative decoding acceptance and scheduling
128	Prefix reuse and long-context optimization
129	Disaggregated prefill and decode
130	Attention backends and kernel selection
131	Mixture-of-Experts inference
132	Expert routing and load balancing
133	MLA and hybrid model architectures
134	Structured output and constrained decoding

PRACTICE MILESTONE Optimize a serving workload while measuring latency, throughput, memory and output quality.

PHASE 08 / 10

Multi-GPU & Distributed Inference

Scale beyond one device and reason about communication costs.

Distributed GPU computing

135	GPU interconnects: PCIe, NVLink, NVSwitch
136	NCCL communication primitives
137	All-reduce, all-gather, reduce-scatter and all-to-all
138	Data parallelism
139	Tensor parallelism
140	Pipeline parallelism
141	Expert parallelism
142	Context/sequence parallelism
143	Combined multi-dimensional parallelism
144	Communication-computation overlap
145	RDMA and InfiniBand
146	Multi-node model serving
147	Distributed KV-cache management
148	Multi-GPU performance profiling

PRACTICE MILESTONE Deploy and benchmark a multi-GPU inference workload on rented cloud hardware.

PHASE 09 / 10

Compilers & Accelerator Programming

Understand how high-level tensor programs become machine operations.

ML compiler systems

149	Compiler design fundamentals
150	Intermediate representations (IR)
151	LLVM and MLIR fundamentals
152	CUDA PTX and instruction-level concepts
153	Triton compiler architecture
154	PyTorch Dynamo and torch.compile
155	TorchInductor and custom operators
156	Kernel fusion and graph transformations

Beyond NVIDIA GPUs

157	JAX and XLA execution fundamentals
158	TPU architecture and memory hierarchy
159	Pallas kernel programming
160	AMD ROCm and HIP fundamentals
161	AWS Trainium and Neuron fundamentals
162	Cross-accelerator numerical correctness and benchmarking

PRACTICE MILESTONE Investigate a compilation trace or compare a kernel across compiler paths / accelerators.

PHASE 10 / 10

Frontier AI Infrastructure

Practice research-minded systems engineering and high-impact reliability work.

Large-scale serving systems

163	GPU fleet scheduling and resource allocation
164	Kubernetes and GPU operators
165	Model placement, autoscaling and capacity planning
166	Multi-region inference routing
167	Load shedding, overload protection and fault tolerance
168	GPU failure diagnosis and recovery
169	Cost-per-token optimization
170	Model rollout, canary deployments and rollback
171	Performance qualification and regression CI
172	Reproducible experiments and benchmark provenance

Advanced engineering and research

173	Read and reproduce inference-performance papers
174	Build automated kernel correctness verifiers
175	Automated optimization and kernel-search systems
176	Profile new model architectures and execution patterns
177	Investigate performance regressions in real runtimes
178	Design and validate an original runtime improvement
179	Contribute a meaningful change to vLLM, SGLang or Triton
180	Write technical reports explaining measurements and trade-offs

PRACTICE MILESTONE Publish an original performance study and attempt a meaningful reviewed open-source contribution.

Free technical references

Use primary documentation and source repositories alongside the roadmap. Focus on concepts and experiments rather than memorizing framework flags, which may change over time.

Build a Large Language Model (From Scratch) - source code

PyTorch tutorials and documentation

NVIDIA CUDA C++ Programming Guide

NVIDIA Nsight Compute profiler

vLLM documentation

SGLang documentation

Triton official tutorials

GPU MODE open learning community

JAX Pallas documentation

A practical note

Not all inference-engineering roles need equal depth in kernels, distributed infrastructure, compilers, and accelerators. The roadmap provides a broad foundation followed by deeper specializations. Real proficiency comes from implementing, debugging, benchmarking, and explaining systems - not from completing the topic list alone.